ChatGPT gpt-5-3

👤 DEV 5 — DATABASE SCHEMA DESIGN (SynapseSignal: PostgreSQL + Redis Hybrid)

🧠 DESIGN PRINCIPLES

You are building a **hybrid system**:

- **PostgreSQL → persistent, relational, analytical**
- **Redis → real-time, fast-access state**

🔷 WHAT GOES WHERE

Data Type	Storage
Real-time signal state	Redis
Live vehicle/emergency tracking	Redis
Historical traffic data	PostgreSQL
Route + corridor logs	PostgreSQL
Intersection + road network	PostgreSQL

🔷 POSTGRESQL SCHEMA (CORE SYSTEM)

🔹 1. INTERSECTIONS TABLE

```sql
CREATE TABLE intersections (
    intersection_id SERIAL PRIMARY KEY,
    name TEXT,
    latitude DOUBLE PRECISION,
    longitude DOUBLE PRECISION
);
```

2. LANES TABLE

```sql
CREATE TABLE lanes (
    lane_id SERIAL PRIMARY KEY,
    intersection_id INT REFERENCES intersections(intersection_id),
    direction TEXT,
    sector_id TEXT
);
```

3. ROADS (EDGES)

```sql
CREATE TABLE roads (
    road_id SERIAL PRIMARY KEY,
    from_intersection INT REFERENCES intersections(intersection_id),
    to_intersection INT REFERENCES intersections(intersection_id),
    distance FLOAT
);
```

4. TRAFFIC STATES (HISTORICAL)

```sql
CREATE TABLE traffic_states (
    state_id SERIAL PRIMARY KEY,
    intersection_id INT,
    timestamp TIMESTAMP,
    total_vehicles INT,
    avg_speed FLOAT,
    congestion_level TEXT
);
```

5. LANE METRICS (FLOW-AWARE)

```sql
```

```sql
CREATE TABLE lane_metrics (
    id SERIAL PRIMARY KEY,
    lane_id INT REFERENCES lanes(lane_id),
    timestamp TIMESTAMP,
    in_density FLOAT,
    out_density FLOAT,
    capacity FLOAT
);
```

🔹 6. EMERGENCY EVENTS

```sql
CREATE TABLE emergency_events (
    event_id SERIAL PRIMARY KEY,
    vehicle_type TEXT,
    start_time TIMESTAMP,
    end_time TIMESTAMP,
    origin_intersection INT,
    destination_intersection INT
);
```

🔹 7. ROUTES TABLE

```sql
CREATE TABLE routes (
    route_id SERIAL PRIMARY KEY,
    event_id INT REFERENCES emergency_events(event_id),
    total_distance FLOAT,
    estimated_time FLOAT
);
```

🔹 8. ROUTE PATH (RELATION TABLE)

```sql
CREATE TABLE route_nodes (
    id SERIAL PRIMARY KEY,
    route_id INT REFERENCES routes(route_id),
    intersection_id INT REFERENCES intersections(intersection_id),
```

```sql
    sequence_order INT
);
```

🔹 9. CORRIDOR LOGS

```sql
CREATE TABLE corridor_logs (
    id SERIAL PRIMARY KEY,
    route_id INT REFERENCES routes(route_id),
    intersection_id INT,
    green_start TIMESTAMP,
    green_end TIMESTAMP
);
```

🔹 10. SIGNAL DECISIONS LOG

```sql
CREATE TABLE signal_logs (
    id SERIAL PRIMARY KEY,
    intersection_id INT,
    timestamp TIMESTAMP,
    selected_sector TEXT,
    reason TEXT
);
```

🔷 REDIS SCHEMA (REAL-TIME STATE)

🔹 1. LIVE INTERSECTION STATE

```text
key: intersection:{id}
value:
{
  lanes: [...],
  sectors: [...],
  signal_state: ...
```

```
}
```

🔹 2. EMERGENCY VEHICLE STATE

```text
key: emergency:{id}
value:
{
  lat: ...,
  lon: ...,
  velocity: ...,
  route_id: ...
}
```

🔹 3. ACTIVE CORRIDOR

```text
key: corridor:active
value:
{
  route: [...],
  current_position: ...,
  next_intersection: ...
}
```

🔹 4. LIVE SIGNAL STATE

```text
key: signal:{intersection_id}
value:
{
  sector: "...",
  state: "GREEN"
}
```

🔷 RELATIONSHIPS (CRITICAL)

🔗 CORE RELATION MAP

```text
intersections
     ↓
lanes (many per intersection)
     ↓
lane_metrics (time series)

intersections
     ↓
roads (graph edges)

emergency_events
     ↓
routes
     ↓
route_nodes (path)

routes
     ↓
corridor_logs

intersections
     ↓
signal_logs
```

🔷 IMPORTANT JOINS

🔹 1. INTERSECTION → LANES

```sql
SELECT * FROM lanes
JOIN intersections
ON lanes.intersection_id = intersections.intersection_id;
```

🔹 2. ROUTE → PATH

```sql
SELECT * FROM route_nodes
JOIN routes
ON route_nodes.route_id = routes.route_id;
```

🔹 3. INTERSECTION → TRAFFIC STATE

```sql
SELECT * FROM traffic_states
WHERE intersection_id = ?;
```

🔹 4. LANE → METRICS

```sql
SELECT * FROM lane_metrics
WHERE lane_id = ?;
```

🔹 5. ROUTE → CORRIDOR EXECUTION

```sql
SELECT * FROM corridor_logs
WHERE route_id = ?;
```

🔷 WHY THIS DESIGN WORKS

✅ NORMALIZED STRUCTURE

- No duplication
- Clean relationships

✅ SUPPORTS REAL-TIME + HISTORY

- Redis → live

- PostgreSQL → analytics

✅ SUPPORTS YOUR SYSTEM FEATURES

Feature	Supported By
Flow-based control	lane_metrics
Sector logic	lanes table
Routing	roads + route_nodes
Corridor	corridor_logs
Map UI	intersections + roads
Emergency tracking	emergency_events

🧠 FINAL INSIGHT

> This database is not just storage —
> it is the **memory + structure of your entire traffic system**

🚀 FINAL RESULT

✔ Fully relational schema
✔ Supports real-time + analytics
✔ Clean joins and dependencies
✔ Compatible with all Dev layers

If you want next:
I can **generate SQL + sample queries + API layer design (Dev 5 execution)**

ChatGPT gpt-5-3

👤 DEV 5 — PLAN (Database Layer Execution + Implementation Guide | SynapseSignal)

🔷 ROLE

Design, implement, and maintain the **Data Persistence & Retrieval Layer**:

- PostgreSQL → structured, historical, relational data
- Redis → real-time system state
- Ensure:
  - consistency
  - performance
  - clean data contracts with Dev 2, 3, 4

🔷 PHASED EXECUTION PLAN (HOW TO BUILD)

🔹 PHASE 1 — DATABASE ENVIRONMENT SETUP

🎯 Goal

Get PostgreSQL + Redis running locally

⚙️ Steps

1. Install PostgreSQL

```bash
sudo apt install postgresql postgresql-contrib
```

2. Create database

```sql
CREATE DATABASE synapsesignal;
```

3. Install Redis

```bash
sudo apt install redis-server
```

4. Start services

```bash
```

```
sudo service postgresql start
sudo service redis-server start
```

✅ Deliverable

✔ Running PostgreSQL + Redis

🔹 PHASE 2 — SCHEMA CREATION (POSTGRESQL)

🎯 Goal

Create all core tables

⚙️ Steps

1. Connect to DB

```bash
psql -d synapsesignal
```

2. Create tables (in order):

- intersections
- lanes
- roads
- traffic_states
- lane_metrics
- emergency_events
- routes
- route_nodes
- corridor_logs
- signal_logs

⚠️ Important

- Create **intersections first** (dependency root)
- Then dependent tables

✅ Deliverable

✔ Full schema created

🔹 PHASE 3 — DATA RELATION VALIDATION

🎯 Goal

Ensure all foreign keys and joins work

⚙️ Steps

Test queries:

```sql
SELECT * FROM lanes
JOIN intersections
ON lanes.intersection_id = intersections.intersection_id;
```

```sql
SELECT * FROM route_nodes
JOIN routes
ON route_nodes.route_id = routes.route_id;
```

✅ Deliverable

✔ Verified relationships

🔹 PHASE 4 — REDIS STRUCTURE SETUP

🎯 Goal

Define real-time data keys

⚙️ Steps

Use Redis CLI:

```bash
redis-cli
```

Define structures:

```bash
SET intersection:1 "{...}"
SET signal:1 "{...}"
SET emergency:1 "{...}"
```

🧠 Key Design

- Use:
  - `intersection:{id}`
  - `signal:{id}`
  - `corridor:active`

✅ Deliverable

✔ Real-time state structure

🔹 PHASE 5 — BACKEND DB INTEGRATION

🎯 Goal

Connect DB with backend (Dev 3)

⚙️ Steps

Using Python:

```python
import psycopg2
import redis

# PostgreSQL
conn = psycopg2.connect(
    dbname="synapsesignal",
    user="postgres",
    password="yourpassword"
)

# Redis
```

```
r = redis.Redis(host='localhost', port=6379, db=0)
```

✅ Deliverable

✔ Backend connected to DB

🔹 PHASE 6 — DATA INSERTION PIPELINES

🎯 Goal

Store system outputs

⚙️ What to Store

From Dev 2:

- traffic_states
- lane_metrics

From Dev 3:

- signal_logs
- corridor_logs
- routes

Example:

```sql
INSERT INTO traffic_states (
    intersection_id, timestamp, total_vehicles
)
VALUES (1, NOW(), 25);
```

✅ Deliverable

✔ Data flowing into DB

🔹 PHASE 7 — REDIS + POSTGRES SYNC STRATEGY

🎯 Goal

Maintain consistency

⚙️ Strategy

- Write to:
  - Redis → immediate
  - PostgreSQL → async

Example Flow:

```text
Dev 3 decision
    ↓
Write → Redis (instant)
    ↓
Write → PostgreSQL (log)
```

✅ Deliverable

✔ Dual-write system

🔹 PHASE 8 — QUERY LAYER (FOR DEV 4)

🎯 Goal

Serve frontend data

⚙️ Build APIs:

- Get intersections
- Get routes
- Get signal states

Example Query:

```sql
```

```sql
SELECT i.name, s.selected_sector
FROM intersections i
JOIN signal_logs s
ON i.intersection_id = s.intersection_id;
```

✅ Deliverable

✔ Query-ready backend

🔹 PHASE 9 — PERFORMANCE OPTIMIZATION

🎯 Goal

Ensure fast queries

⚙️ Steps

Add indexes:

```sql
CREATE INDEX idx_intersection
ON traffic_states(intersection_id);
```

Optimize:

- frequent joins
- time-series queries

✅ Deliverable

✔ Optimized DB

🔹 PHASE 10 — DATA VALIDATION & CONSTRAINTS

🎯 Goal

Prevent bad data

⚙️ Steps

Add constraints:

```sql
ALTER TABLE lane_metrics
ADD CONSTRAINT positive_density
CHECK (in_density >= 0);
```

✅ Deliverable

✔ Clean data enforcement

🔹 PHASE 11 — LOGGING & HISTORY MANAGEMENT

🎯 Goal

Maintain long-term data

⚙️ Steps

- Store:
  - all signal decisions
  - all corridor events
- Use timestamps

Optional:

- Archive old data

✅ Deliverable

✔ Complete system history

🔹 PHASE 12 — TESTING & VALIDATION

🎯 Goal

Ensure system reliability

⚙️ Steps

Test:

- inserts
- joins
- API responses
- Redis reads

Edge cases:

- missing data
- high load

✅ Deliverable

✔ Stable DB system

🔷 HOW EVERYTHING CONNECTS

```text
Dev 2 → traffic + lane metrics → PostgreSQL

Dev 3 → decisions + corridors → Redis (live)
                                  ↓
                               PostgreSQL (log)

Dev 4 → reads:
   - Redis (live state)
   - PostgreSQL (history)
```

🧠 FINAL INSIGHT

> Redis = **current brain state**
> PostgreSQL = **long-term memory**

🚀 FINAL RESULT

✔ Fully implemented database system
✔ Supports real-time + analytics

✔ Clean integration across all devs
✔ Scalable and production-ready foundation

If you want next:
I can generate **Dev 5 master prompts + guided learning (same format as others)**

ChatGPT gpt-5-3

👤 DEV 5 — COMPLETE ENVIRONMENT SETUP + EXECUTION PLAN (SynapseSignal)

🔷 PART 1 — FULL SYSTEM SETUP (FROM ZERO)

🔹 STEP 0 — GET LINUX (WSL RECOMMENDED FOR YOU)

🎯 **Why**

You need Linux because:

- `sudo` commands
- better compatibility for PostgreSQL + Redis
- smoother backend execution

🔹 OPTION A — WSL (BEST FOR WINDOWS)

1. Install WSL

Open PowerShell (Admin):

```bash
wsl --install
```

---

2. Restart system

---

3. Install Ubuntu

From Microsoft Store → install **Ubuntu**

---

4. First Launch

- Set username + password
- This password = your `sudo` password

---

5. Update system

```bash
sudo apt update
sudo apt upgrade -y
```

🔹 OPTION B — DUAL BOOT (ONLY IF ADVANCED)

Not needed for your current setup → skip

🔷 PART 2 — BASIC LINUX ENVIRONMENT SETUP

🔹 Install Essential Tools

```bash
sudo apt install -y build-essential curl wget git
```

🔹 Verify Python

```bash
python3 --version
```

If not:

```bash
sudo apt install python3 python3-pip -y
```

🔹 Set PATH (IMPORTANT)

Open bash config:

```bash
nano ~/.bashrc
```

Add:

```bash
export PATH=$PATH:/usr/local/bin
```

Save:

- `CTRL + X`
- `Y`
- Enter

Apply:

```bash
source ~/.bashrc
```

🔷 PART 3 — POSTGRESQL SETUP (FULL)

🔹 Install PostgreSQL

```bash
sudo apt install postgresql postgresql-contrib -y
```

🔹 Start Service

```bash
sudo service postgresql start
```

Switch to PostgreSQL User

```bash
sudo -i -u postgres
```

Open PostgreSQL CLI

```bash
psql
```

Create Database

```sql
CREATE DATABASE synapsesignal;
```

Create User (IMPORTANT)

```sql
CREATE USER synapse_user WITH PASSWORD 'strongpassword';
ALTER ROLE synapse_user SET client_encoding TO 'utf8';
ALTER ROLE synapse_user SET default_transaction_isolation TO 'read committed';
ALTER ROLE synapse_user SET timezone TO 'UTC';
GRANT ALL PRIVILEGES ON DATABASE synapsesignal TO synapse_user;
```

Exit

```sql
\q
exit
```

Test Connection

```bash
psql -U synapse_user -d synapsesignal -h localhost
```

🔷 PART 4 — REDIS SETUP (FULL)

🔹 Install Redis

```bash
sudo apt install redis-server -y
```

🔹 Start Redis

```bash
sudo service redis-server start
```

🔹 Test Redis

```bash
redis-cli
```

Then:

```bash
ping
```

Expected:

```text
PONG
```

🔹 Exit

```bash
exit
```

🔷 PART 5 — PYTHON DB CONNECTIVITY SETUP

🔹 Install Required Libraries

```bash
pip install psycopg2-binary redis
```

🔹 Test Connection Script

```python
import psycopg2
import redis

# PostgreSQL
conn = psycopg2.connect(
    dbname="synapsesignal",
    user="synapse_user",
    password="strongpassword",
    host="localhost"
)

print("PostgreSQL connected")

# Redis
r = redis.Redis(host='localhost', port=6379, db=0)
print(r.ping())
```

🔷 PART 6 — PROJECT DIRECTORY STRUCTURE

```bash
```

```
mkdir synapsesignal
cd synapsesignal

mkdir backend database scripts logs
```

🔷 PART 7 — DATABASE EXECUTION PLAN (AFTER SETUP)

🔹 PHASE 1 — CREATE SCHEMA

Run SQL:

```bash
psql -U synapse_user -d synapsesignal
```

Then create all tables:

- intersections
- lanes
- roads
- traffic_states
- lane_metrics
- emergency_events
- routes
- route_nodes
- corridor_logs
- signal_logs

🔹 PHASE 2 — INSERT BASE DATA

Insert:

- intersections (with lat/lon)
- roads (graph edges)
- lanes

🔹 PHASE 3 — CONNECT WITH DEV 2

Store:

- lane_metrics
- traffic_states

🔹 PHASE 4 — CONNECT WITH DEV 3

Store:

- routes
- corridor_logs
- signal_logs

🔹 PHASE 5 — REDIS REAL-TIME STATE

Store:

```text
intersection:{id}
signal:{id}
emergency:{id}
corridor:active
```

🔹 PHASE 6 — BUILD QUERY LAYER

Backend APIs:

- get intersections
- get route
- get signals

🔹 PHASE 7 — OPTIMIZATION

Add indexes:

```sql
CREATE INDEX idx_lane_metrics ON lane_metrics(lane_id);
CREATE INDEX idx_traffic_time ON traffic_states(timestamp);
```

🔹 PHASE 8 — TESTING

Test:

- DB inserts
- joins
- Redis reads
- API responses

🔷 FINAL SYSTEM FLOW

```text
WSL (Linux)
    ↓
PostgreSQL ← persistent storage
Redis ← real-time state
    ↓
Python Backend (Dev 3)
    ↓
Frontend (Dev 4)
```

🧠 FINAL INSIGHT

> You are not just installing tools —
> you are setting up the **data backbone of SynapseSignal**

🚀 FINAL RESULT

✔ Linux environment ready
✔ PostgreSQL fully configured
✔ Redis real-time system ready
✔ Python connectivity working
✔ Database execution plan defined

If you want next:
I can generate **Dev 5 master prompts + guided learning (like other devs)**

ChatGPT gpt-5-3

👤 DEV 5 — MASTER PROMPTS (Database Layer | SynapseSignal)

1️⃣ MASTER PROMPT — POSTGRESQL SCHEMA CREATION

You are a database engineer.

Task:
Generate complete SQL schema for SynapseSignal.

You must create tables:

- intersections
- lanes
- roads
- traffic_states
- lane_metrics
- emergency_events
- routes
- route_nodes
- corridor_logs
- signal_logs

Requirements:

- Proper PRIMARY KEYS
- FOREIGN KEY relationships
- Correct data types

Output:

- Executable SQL script (ordered correctly)

Constraints:

- No circular dependencies
- Must follow relational integrity

2️⃣ MASTER PROMPT — INITIAL DATA SEEDING

You are populating base system data.

Task:
Insert initial data for:

- intersections (with lat/lon)
- roads (graph edges)
- lanes

You must:

- Maintain consistency
- Ensure all foreign keys valid

Output:

```sql
INSERT INTO intersections ...
INSERT INTO roads ...
```

Constraints:

- No orphan references
- Realistic sample data

3️⃣ MASTER PROMPT — REDIS KEY DESIGN

You are designing real-time storage.

Task:
Define Redis key-value structure.

You must define keys:

- intersection:{id}
- signal:{id}
- emergency:{id}
- corridor:active

Output format:

```json
```

```
{
  "key": "intersection:1",
  "value": {...}
}
```

Constraints:

- Must support real-time updates
- No redundant keys

4️⃣ MASTER PROMPT — PYTHON DATABASE CONNECTIVITY

You are building backend DB layer.

Task:
Create Python module to connect PostgreSQL + Redis.

You must:

- Use:
  - psycopg2
  - redis
- Implement:
  - connection pooling
  - error handling

Output:

- Python code

Constraints:

- No hardcoded credentials
- Must be reusable

5️⃣ MASTER PROMPT — DATA INSERTION PIPELINES

You are designing data ingestion.

Task:
Store data from Dev 2 and Dev 3.

You must implement:

- Insert traffic_states

- Insert lane_metrics
- Insert routes and logs

Output:

- Python functions

Constraints:

- No duplicate inserts
- Must handle high-frequency writes

6️⃣ MASTER PROMPT — REDIS + POSTGRES SYNC LOGIC

You are designing hybrid storage logic.

Task:
Implement dual-write system.

You must:

- Write:
  - Redis (real-time)
  - PostgreSQL (persistent)
- Ensure:
  - consistency
  - fallback if one fails

Output:

- Python logic

Constraints:

- No data loss
- Must be fault-tolerant

7️⃣ MASTER PROMPT — QUERY LAYER (API SUPPORT)

You are designing query system.

Task:
Create queries for frontend/backend.

You must implement:

- Get intersections
- Get route paths
- Get signal history
- Get traffic trends

Output:

- SQL queries + Python wrappers

Constraints:

- Must be optimized
- No unnecessary joins

8️⃣ MASTER PROMPT — INDEXING & PERFORMANCE OPTIMIZATION

You are optimizing database.

Task:
Add indexes and optimize queries.

You must:

- Identify:
  - frequent queries
- Add indexes:
  - intersection_id
  - timestamp

Output:

```sql
CREATE INDEX ...
```

Constraints:

- No redundant indexes
- Must improve performance

9️⃣ MASTER PROMPT — DATA VALIDATION & CONSTRAINTS

You are enforcing data integrity.

Task:
Add constraints.

You must:

- Enforce:
  - non-negative density
  - valid timestamps
- Add:
  - CHECK constraints

Output:

```sql
ALTER TABLE ...
```

Constraints:

- Must prevent invalid data
- No overly strict constraints

🔟 MASTER PROMPT — LOGGING SYSTEM

You are building system logs.

Task:
Store all decisions and events.

You must log:

- signal decisions
- corridor execution
- emergency events

Output:

- Insert queries

Constraints:

- Must include timestamps
- Must be queryable

1️⃣1️⃣ MASTER PROMPT — TIME-SERIES DATA HANDLING

You are managing temporal data.

Task:
Efficiently store and query time-series data.

You must:

- Partition or optimize:
  - traffic_states
  - lane_metrics

Output:

- SQL strategy

Constraints:

- Must support large-scale data
- No slow queries

1️⃣2️⃣ MASTER PROMPT — TESTING & VALIDATION

You are testing database system.

Task:
Validate DB correctness.

You must test:

- inserts
- joins
- queries
- Redis reads

Output:

- Test scripts

Constraints:

- Must cover edge cases
- No partial testing

🔷 DEV 5 MASTER PROMPT OUTCOME

- Fully structured PostgreSQL schema
- Real-time Redis key system
- Backend DB connectivity
- Reliable data ingestion pipelines
- Dual-write consistency system
- Optimized query layer
- Indexed and performant DB
- Strong data validation rules
- Complete logging system
- Scalable time-series handling
- Fully tested database layer

🧠 FINAL INSIGHT

> Dev 5 is not just storage —
> it is the **memory, reliability, and traceability of SynapseSignal**

✔ Ready for **Guided Learning (if needed)**

ChatGPT gpt-5-3

👤 DEV 5 — GUIDED LEARNING (FULL SETUP → DATABASE SYSTEM | SynapseSignal)

🔷 PHASE 0 — WHY THIS SETUP EXISTS

🧠 What You Are Learning

You are learning **why a hybrid database (PostgreSQL + Redis) is necessary**.

⚙️ Reality of Your System

- Traffic system = real-time + historical
- One database cannot efficiently do both

🧠 Core Split

- Redis → **live brain (fast, volatile)**
- PostgreSQL → **long-term memory (stable, queryable)**

🧱 Mental Model

You need both speed and memory

🔷 PHASE 1 — LINUX ENVIRONMENT (WSL)

🧠 What You Are Learning

You are learning **how real backend systems run in Linux environments**.

⚙️ What Is Happening

- You install WSL (Linux inside Windows)
- Use:
  - `sudo`
  - package manager (`apt`)

🔍 What to Notice

- Linux gives:
  - control
  - compatibility
- Most production systems run on Linux

🧱 Mental Model

Linux = foundation of backend systems

🔷 PHASE 2 — POSTGRESQL SETUP

🧠 What You Are Learning

You are learning **relational data modeling and persistence**.

⚙️ What Is Happening

- Install PostgreSQL
- Create:
  - database
  - user
- Establish connection

🔍 What to Notice

- Tables define system structure
- Relationships define system logic

🧱 Mental Model

PostgreSQL = structured memory of system

🔷 PHASE 3 — REDIS SETUP

🧠 What You Are Learning

You are learning **real-time state storage**.

⚙️ What Is Happening

- Redis stores:
  - current signals
  - live vehicle position
- Key-value system (no joins)

🔍 What to Notice

- Extremely fast
- No history

🧱 Mental Model

Redis = current snapshot of reality

🔷 PHASE 4 — PYTHON DB CONNECTIVITY

🧠 What You Are Learning

You are learning **how backend talks to databases**.

⚙️ What Is Happening

- Connect Python → PostgreSQL
- Connect Python → Redis

🔍 What to Notice

- Connections must be stable
- Errors must be handled

🧱 Mental Model

Backend = bridge between logic and storage

🔷 PHASE 5 — SCHEMA CREATION

🧠 What You Are Learning

You are learning **how to model the real world in tables**.

⚙️ What Is Happening

You define:

- intersections
- roads
- lanes
- routes

🔍 What to Notice

- Each table = real-world entity
- Foreign keys = relationships

🧱 Mental Model

Database schema = system blueprint

🔷 PHASE 6 — RELATIONSHIPS

🧠 What You Are Learning

You are learning **how entities connect**.

⚙️ What Is Happening

```text
intersection → lanes → lane_metrics
intersection → roads → graph
routes → route_nodes → path
```

🔍 What to Notice

- Bad relationships → bad queries
- Proper joins = powerful system

🧱 Mental Model

Relationships define intelligence of database

🔷 PHASE 7 — DATA INSERTION PIPELINES

🧠 What You Are Learning

You are learning **how system outputs become stored data**.

⚙️ What Is Happening

From Dev 2:

- lane_metrics
- traffic_states

From Dev 3:

- routes
- signal_logs
- corridor_logs

🔍 What to Notice

- Data is continuously written
- Must handle high frequency

🧱 Mental Model

Database is continuously evolving state

🔷 PHASE 8 — REDIS + POSTGRES SYNC

🧠 What You Are Learning

You are learning **dual-write architecture**.

⚙️ What Is Happening

```text
```

```
Decision
    ↓
Redis (instant)
    ↓
PostgreSQL (logged)
```

🔍 What to Notice

- Redis gives speed
- PostgreSQL gives reliability

🧱 Mental Model

Fast now + stored forever

🔷 PHASE 9 — QUERY LAYER

🧠 What You Are Learning

You are learning **how frontend retrieves data efficiently**.

⚙️ What Is Happening

- Queries fetch:
  - intersections
  - routes
  - signals

🔍 What to Notice

- Bad queries → slow system
- Good queries → real-time UX

🧱 Mental Model

Queries = information access layer

🔷 PHASE 10 — INDEXING & PERFORMANCE

🧠 What You Are Learning

You are learning **how databases scale**.

⚙️ What Is Happening

- Add indexes to:
  - frequently queried columns

🔍 What to Notice

- Without index → full table scan
- With index → fast lookup

🧱 Mental Model

Indexes = shortcuts in database

🔷 PHASE 11 — DATA VALIDATION

🧠 What You Are Learning

You are learning **data correctness enforcement**.

⚙️ What Is Happening

- Add constraints:
  - no negative density
  - valid timestamps

🔍 What to Notice

- Prevents corrupt data
- Protects system integrity

🧱 Mental Model

Database must reject bad data

🔷 PHASE 12 — LOGGING SYSTEM

🧠 What You Are Learning

You are learning **system observability**.

⚙️ What Is Happening

- Store:
  - decisions
  - events
  - corridor actions

🔍 What to Notice

- Logs allow:
  - debugging
  - analysis

🧱 Mental Model

Logs = system memory of actions

🔷 PHASE 13 — TIME-SERIES HANDLING

🧠 What You Are Learning

You are learning **how to manage continuously growing data**.

⚙️ What Is Happening

- Store:

    - traffic over time
- Optimize:
    - queries

🔍 What to Notice

- Data grows fast
- Needs efficient storage

🧱 Mental Model

Time-series = evolving system history

🔷 PHASE 14 — TESTING

🧠 What You Are Learning

You are learning **system reliability validation**.

⚙️ What Is Happening

- Test:
    - inserts
    - joins
    - Redis reads

🔍 What to Notice

- Edge cases matter
- Failure must be handled

🧱 Mental Model

Test before trust

🔷 FINAL LEARNING OUTCOME

You now understand:

- Why Linux is used for backend systems
- How PostgreSQL stores structured system data
- How Redis handles real-time state
- How backend connects to databases
- How schema models real-world traffic
- How relationships enable powerful queries
- How data flows from Dev 2 and Dev 3
- How Redis and PostgreSQL work together
- How frontend retrieves data
- How indexing improves performance
- How validation ensures correctness
- How logging enables traceability
- How time-series data is handled
- How to test and validate the entire system

🧠 FINAL INSIGHT

> Dev 5 transforms raw system activity
> into **structured memory, real-time state, and reliable history**

✔ SynapseSignal is now:

- Fully structured
- Fully connected
- Ready for execution end-to-end